

Programare Orientată Pe Obiecte

Curs 12 - Java I/O și Serializare

Decembrie 2025

Agenda

- 1 Introducere
- 2 Citirea și Scrierea
- 3 Excepții I/O
- 4 Exerciții 1
- 5 Serializarea
- 6 Exerciții 2

Introducere

Ce este Java I/O?

Input/Output = comunicarea programului cu lumea exterioară

Surse de date (Input):

- Fișiere de pe disc
- Tastatura (System.in)
- Rețea (sockets)
- Alte programe

Destinații (Output):

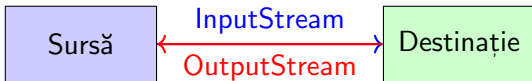
- Fișiere de pe disc
- Ecran/Consolă (System.out)
- Rețea
- Alte programe

Pachetul principal: `java.io`

Conceptul de Stream (Flux)

Sistemul de I/O în Java este folosit pentru a citi și scrie date.

- Se bazează pe fluxuri (streams):
- Un **stream** = o secvență de date care "curge" între sursă și destinație.
 - Fluxuri de intrare (Input Streams) pentru citirea datelor.
 - Fluxuri de ieșire (Output Streams) pentru scrierea datelor.



Două categorii principale:

- **Byte Streams** - pentru date binare (imagini, audio, video)
- **Character Streams** - pentru text (folosesc Unicode)

Java oferă clase în pachetul `java.io` pentru operații I/O de bază.

- `InputStream` și `OutputStream` sunt clase abstracte pentru fluxuri de byte.
- `Reader` și `Writer` sunt clase abstracte pentru fluxuri de caractere.

Ierarhia claselor I/O

Tip	Input	Output
Bytes (date binare)	InputStream FileInputStream BufferedInputStream	OutputStream FileOutputStream BufferedOutputStream
Caractere (text)	Reader FileReader BufferedReader	Writer FileWriter BufferedWriter

Regulă simplă:

- Text (CSV, JSON, cod sursă) → Reader/Writer
- Binar (imagini, PDF, .ser) → InputStream/OutputStream

Acces secvențial vs. aleator

Acces secvențial

- Citire/scriere de la început la sfârșit
- Nu poți "sări" la o poziție
- Majoritatea stream-urilor
- Ex: `BufferedReader`

Acces aleator

- Poți accesa orice poziție
- Folosește `seek()`
- Doar fișiere pe disc
- Ex: `RandomAccessFile`

`System.in` și `System.out` suportă doar acces secvențial!

Tipuri de formate de stocare

- **Formatat:**

- Datele sunt stocate într-un format ușor de citit de către oameni (e.g., text).
- Exemple: fișiere CSV, JSON, XML.

- **Neformatat:**

- Datele sunt stocate într-un format binar, optimizat pentru mașini.
- Exemplu: imagini, fișiere audio/video.

Citirea și Scrierea

Citirea unui fișier text - metoda clasică

```
1 import java.io.*;
2 public class CitireFisier {
3     public static void main(String[] args) {
4         // try-with-resources: inchide automat fisierul
5         try (BufferedReader br = new BufferedReader(
6             new FileReader("date.txt"))) {
7             String linie;
8             while ((linie = br.readLine()) != null) {
9                 System.out.println(linie);
10            }
11        } catch (FileNotFoundException e) {
12            System.out.println("Fisierul nu exista!");
13        } catch (IOException e) {
14            System.out.println("Eroare:" + e.getMessage());
15        }
16    }
```

De ce BufferedReader? Citește în blocuri mari → mai eficient decât caracter cu caracter.

Scrierea într-un fișier text

```
1 import java.io.*;
2 public class ScriereFisier {
3     public static void main(String[] args) {
4         try (PrintWriter pw = new PrintWriter(
5             new FileWriter("output.txt"))) {
6
7             pw.println("Prima linie");
8             pw.println("A doua linie");
9             pw.printf("Numar: %d, Text: %s\n", 42, "test");
10
11         } catch (IOException e) {
12             System.out.println("Eroare la scriere: " + e.getMessage
13                 ());
14         }
15 }
```

Append (adaugă la final):

```
new FileWriter("output.txt", true) // true = append mode
```

API-ul modern: java.nio.file (Java 7+)

Citire - tot fișierul dintr-o dată:

```
1 import java.nio.file.*;
2 import java.util.List;
3
4 // Citeste toate liniile intr-o lista
5 List<String> linii = Files.readAllLines(Path.of("date.txt"));
6
7 // Sau tot continutul ca String
8 String continut = Files.readString(Path.of("date.txt"));
```

Scriere - la fel de simplu:

```
1 // Scrie o lista de linii
2 Files.write(Path.of("output.txt"), linii);
3
4 // Sau un String direct
5 Files.writeString(Path.of("output.txt"), "Continut nou");
```

Când să folosești NIO: Fișiere mici-medii, când vrei cod concis.

Când NU: Fișiere foarte mari (nu încap în memorie).

Comparație: Când folosim ce?

Situație	Clasă recomandată
Citire text linie cu linie	BufferedReader
Fișier text mic/mediu	Files.readAllLines()
Input de la tastatură	Scanner
Parsing date structurate	Scanner
Scriere text formatat	PrintWriter
Fișiere binare	FileInputStream/FileOutputStream
Obiecte Java (persistență)	ObjectOutputStream
Acces la poziție specifică	RandomAccessFile

Clasa java.io.File

File = reprezentarea unui fișier sau director (nu conținutul!).

```
1 File f = new File("date.txt");
2
3 // Verificari
4 f.exists();           // exista?
5 f.isFile();           // e fisier?
6 f.isDirectory();     // e director?
7 f.canRead();          // poate fi citit?
8 f.length();           // dimensiune in bytes
9
10 // Operatii
11 f.createNewFile();    // creeaza fisier gol
12 f.delete();           // sterge
13 f.renameTo(new File("nou.txt"));
14
15 // Directoare
16 File dir = new File("poze");
17 dir.mkdir();          // creeaza director
18 String[] continut = dir.list(); // listeaza continut
```

Clasa Scanner - Citire de la tastatură

```
1 import java.util.Scanner;
2
3 public class ExempluScanner {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.print("Nume: ");
8         String nume = sc.nextLine(); // citește linie întreaga
9
10        System.out.print("Varsta: ");
11        int varsta = sc.nextInt(); // citește int
12
13        System.out.print("Salariu: ");
14        double salariu = sc.nextDouble();
15
16        System.out.printf("Salut %s, ai %d ani%n", nume, varsta);
17
18        sc.close();
19    }
20 }
```

Scanner - Citire din fișier

```
1 import java.util.Scanner;
2 import java.io.File;
3 import java.io.FileNotFoundException;
4
5 public class ScannerFisier {
6     public static void main(String[] args) {
7         try {
8             Scanner sc = new Scanner(new File("note.txt"));
9
10            while (sc.hasNextLine()) {
11                String nume = sc.next();           // primul cuvant
12                double nota = sc.nextDouble();    // numarul
13                System.out.println(nume + ": " + nota);
14            }
15
16            sc.close();
17        } catch (FileNotFoundException e) {
18            System.out.println("Fisierul nu exista!");
19        }
20    }
21 }
```

Metodele Scanner

Metodă	Returnează
<code>nextLine()</code>	String (toată linia)
<code>next()</code>	String (până la spațiu)
<code>nextInt()</code>	int
<code>nextDouble()</code>	double
<code>nextBoolean()</code>	boolean
<code>hasNextLine()</code>	true dacă mai sunt linii
<code>hasNextInt()</code>	true dacă urmează un int
<code>hasNext()</code>	true dacă mai sunt tokeni

Atenție la `InputMismatchException`!

Apare când tipul nu corespunde (ex: `nextInt()` când în fișier e text).

Soluție: Folosește `hasNextInt()` înainte de `nextInt()`.

Capcana nextInt() + nextLine()

```
1 Scanner sc = new Scanner(System.in);
2
3 System.out.print("Varsta: ");
4 int varsta = sc.nextInt(); // citeste "25", lasa "\n" in buffer
5
6 System.out.print("Nume: ");
7 String nume = sc.nextLine(); // citeste "\n" ramas -> String gol!
```

Soluția:

```
1 int varsta = sc.nextInt();
2 sc.nextLine(); // "consumam" newline-ul ramas
3 String nume = sc.nextLine(); // acum functioneaza corect
```

Excepții I/O

Excepții frecvente în I/O

- **IOException** - clasa părinte pentru toate excepțiile I/O
- **FileNotFoundException** - fișierul nu există sau nu poate fi deschis
- **EOFException** - s-a ajuns la sfârșitul fișierului neașteptat
- **InputMismatchException** - (Scanner) tipul datelor nu corespunde

Toate sunt checked exceptions (cu excepția `InputMismatchException`).

→ Trebuie tratate cu try-catch sau declarate cu throws.

Pattern: try-with-resources (Java 7+)

Problema: Stream-urile trebuie închise pentru a elibera resursele.

Metoda veche (problematică):

```
1  BufferedReader br = null;
2  try {
3      br = new BufferedReader(new FileReader("f.txt"));
4      // ... folosire
5  } finally {
6      if (br != null) br.close(); // poate arunca exceptie!
7  }
```

Metoda modernă (recomandată):

```
1  try (BufferedReader br = new BufferedReader(new FileReader("f.txt")) {
2      // ... folosire
3  } // br.close() apelat automat, chiar si la exceptie
```

Orice clasă care implementează `AutoCloseable` poate fi folosită aici.

Exerciții 1

Exercițiul 1 — Hello File (scriere simplă)

Exersam `PrintWriter`:

- 1 Creează fișierul `hello.txt`
- 2 Scrie în el linia: `Hello Java I/O!`
- 3 Afișează în consolă: `Am scris cu succes!`

Hint: `new PrintWriter(new FileWriter("hello.txt"))`

Extra (1 minut): folosește `append=true` ca să adaugi, nu să suprascrii.

Soluție 1 — Hello File

```
1 import java.io.*;
2 public class Ex1HelloFile {
3     public static void main(String[] args) {
4         try (PrintWriter pw = new PrintWriter(new FileWriter("hello
5             .txt"))) {
6             pw.println("Hello Java I/O!");
7             System.out.println("Am scris cu succes!");
8         } catch (IOException e) {
9             System.out.println("Eroare: " + e.getMessage());
10        }
11    }
```

Variantă append:

```
new FileWriter("hello.txt", true)
```

Exercițiul 2 — Citire linie cu linie + numerotare

Exersam `BufferedReader` + contor

- 1 Se citește fișierul `mesaj.txt` cu `BufferedReader`
- 2 Afișează fiecare linie ca: `nr: linie`
- 3 Numără câte linii au fost citite (la final)

Soluție 2 — Numerotare linii

```
1 import java.io.*;
2
3 public class Ex2NumerotareLinii {
4     public static void main(String[] args) {
5         int i = 1;
6         try (BufferedReader br = new BufferedReader(new FileReader(
7             "mesaj.txt"))) {
8             String linie;
9             while ((linie = br.readLine()) != null) {
10                 System.out.println(i + ": " + linie);
11                 i++;
12             }
13             System.out.println("Total linii: " + (i - 1));
14         } catch (FileNotFoundException e) {
15             System.out.println("Fisierul nu exista!");
16         } catch (IOException e) {
17             System.out.println("Eroare: " + e.getMessage());
18         }
19 }
```

Exercițiul 3 — Linii / Cuvinte / Caractere

- ❶ Se citește fișierul `text.txt` linie cu linie
- ❷ Determinați:
 - numărul de linii
 - numărul de cuvinte
 - numărul de caractere (fără `\n`)
- ❸ Afișați cele 3 valori

Hint: pentru cuvinte: `linie.trim().split("\\s+")`

Soluție 3 — Linii / Cuvinte / Caractere

```
1 import java.io.*;
2 public class Ex3StatisticiText {
3     public static void main(String[] args) {
4         int linii = 0, cuvinte = 0, caractere = 0;
5         try (BufferedReader br = new BufferedReader(new FileReader(
6             "text.txt"))) {
7             String linie;
8             while ((linie = br.readLine()) != null) {
9                 linii++;
10                caractere += linie.length();
11                String t = linie.trim();
12                if (!t.isEmpty()) {
13                    cuvinte += t.split("\\s+").length;
14                }
15                System.out.println("Linii: " + linii);
16                System.out.println("Cuvinte: " + cuvinte);
17                System.out.println("Caractere: " + caractere);
18            } catch (IOException e) {
19                System.out.println("Eroare: " + e.getMessage());
20            }
21        }
22    }
```

Exercițiul 4 — Scanner: media notelor din fișier

Exersăm: Scanner(File) + parsing

Se dă fișierul `note.txt` (exemplu):

```
Ana 9.50  
Ion 8.75  
Maria 10
```

- 1 Citiți perechi: nume (String) + nota (double)
- 2 Calculați media
- 3 Afișați: Media = ...

Soluție 4 — Media notelor

```
1 import java.io.*; import java.util.*;
2 public class Ex4MediaNote {
3     public static void main(String[] args) {
4         double suma = 0;
5         int n = 0;
6         try (Scanner sc = new Scanner(new File("note.txt"))) {
7             while (sc.hasNext()) {
8                 String nume = sc.next();
9                 if (!sc.hasNextDouble()) { // protectie basic
10                     System.out.println("Linie invalida dupa: " +
11                                     nume);
12                     sc.nextLine();
13                     continue;
14                 }
15                 double nota = sc.nextDouble();
16                 suma += nota;
17                 n++;
18             }
19             if (n > 0) {System.out.print("Media="+ (suma/n));
20             } else {System.out.print("Note invalide.");}
21         } catch (FileNotFoundException e) {
22             System.out.println("Fisierul nu exista!");
23         }
```

Exercițiul 5 — Copiere fișier (input.txt → output.txt)

Exersam: citire + scriere (BufferedReader/PrintWriter)

- 1 Deschideți un fișier `input.txt` pentru citire
- 2 Deschideți fișierul `output.txt` pentru scriere
- 3 Copiați fiecare linie (exact cum e) din `input.txt` în `output.txt`

Extra cerință: la scrierea în fișierul `output` transformați tot textul în MAJUSCULE în fișierul de ieșire.

Soluție 5 — Copiere fișier

```
1 import java.io.*;
2
3 public class Ex5CopiereFisier {
4     public static void main(String[] args) {
5         try (BufferedReader br = new BufferedReader(new FileReader(
6             "input.txt"));
7             PrintWriter pw = new PrintWriter(new FileWriter("
8                 output.txt"))) {
9
10            String linie;
11            while ((linie = br.readLine()) != null) {
12                pw.println(linie); // sau: pw.println(linie.
13                    toUpperCase());
14            }
15            System.out.println("Copiere terminata!");
16        } catch (IOException e) {
17            System.out.println("Eroare: " + e.getMessage());
18        }
19    }
20 }
```

Exercițiul 6 — Tastatură → fișier până la "exit"

Exersare: `Scanner(System.in)` + scriere în fișier

- 1 Citiți mai multe linii de la tastatură
- 2 Scriți fiecare linie în fișierul `inregistrare.txt`
- 3 Când la tastatură se scrie `exit`, opriți scrierea (cuvântul "exit" nu se scrie în fișier)

Soluție 6 — Tastatură → fișier

```
1 import java.io.*;
2 import java.util.*;
3 public class Ex6ConsolaInFisier {
4     public static void main(String[] args) {
5         try (Scanner sc = new Scanner(System.in);
6             PrintWriter pw = new PrintWriter(new FileWriter("
7                 inregistrare.txt", true))) {
8             System.out.println("Scrie linii. Tasteaza 'exit' pentru
9                 stop.");
10            while (true) {
11                String linie = sc.nextLine();
12                if (linie.equalsIgnoreCase("exit")) break;
13                pw.println(linie);
14            }
15            System.out.println("Gata!");
16        } catch (IOException e) {
17            System.out.println("Eroare: " + e.getMessage());
18        }
19    }
20 }
```

Exercițiul 7 – NIO

Exersăm `java.nio.file.Files`

- ❶ Citește tot fișierul `date.txt` cu `Files.readString`
- ❷ Afișează:
 - numărul de caractere
 - primele 50 de caractere (dacă există)

Hint: `content.substring(0, Math.min(50, content.length()))`

Soluție 7 — NIO: readString

```
1 import java.io.IOException;
2 import java.nio.file.*;
3
4 public class Ex7NIOReadString {
5     public static void main(String[] args) {
6         try {
7             String content = Files.readString(Path.of("date.txt"));
8             System.out.println("Caractere: " + content.length());
9
10            int k = Math.min(50, content.length());
11            System.out.println("Primele " + k + " caractere:");
12            System.out.println(content.substring(0, k));
13        } catch (IOException e) {
14            System.out.println("Eroare: " + e.getMessage());
15        }
16    }
17 }
```

Serializarea

Ce este serializarea?

Serializare = conversia unui obiect Java într-un șir de bytes.

Deserializare = reconstruirea obiectului din bytes.



Cazuri de utilizare:

- Salvarea stării unui joc
- Cache-uri persistente
- Trimiterea obiectelor prin rețea
- Backup-uri de date

Cum facem o clasă serializabilă?

```
1 import java.io.Serializable;
2
3 public class Student implements Serializable {
4     private static final long serialVersionUID = 1L;
5
6     private String nume;
7     private int varsta;
8     private double medie;
9
10    public Student(String nume, int varsta, double medie) {
11        this.nume = nume;
12        this.varsta = varsta;
13        this.medie = medie;
14    }
15
16    // getters, setters, toString...
17 }
```

serialVersionUID - identifică versiunea clasei. Dacă modifici clasa, schimbă și acest număr!

Salvarea unui obiect (Serializare)

```
1 import java.io.*;
2
3 public class SalvareStudent {
4     public static void main(String[] args) {
5         Student s = new Student("Ion Popescu", 21, 8.75);
6
7         try (ObjectOutputStream out = new ObjectOutputStream(
8             new FileOutputStream("student.ser"))) {
9
10            out.writeObject(s);
11            System.out.println("Student salvat!");
12
13        } catch (IOException e) {
14            System.out.println("Eroare: " + e.getMessage());
15        }
16    }
17 }
```

Fișierul `student.ser` va conține reprezentarea binară a obiectului.

Încărcarea unui obiect (Deserializare)

```
1 import java.io.*;
2
3 public class IncarcareStudent {
4     public static void main(String[] args) {
5         try (ObjectInputStream in = new ObjectInputStream(
6             new FileInputStream("student.ser"))) {
7
8             Student s = (Student) in.readObject();
9             System.out.println("Student incarcat: " + s);
10
11         } catch (FileNotFoundException e) {
12             System.out.println("Fisierul nu exista!");
13         } catch (ClassNotFoundException e) {
14             System.out.println("Clasa Student nu a fost gasita!");
15         } catch (IOException e) {
16             System.out.println("Eroare I/O: " + e.getMessage());
17         }
18     }
19 }
```

Atenție: `readObject()` returnează `Object` - trebuie cast!

Serializarea colecțiilor

```
1 import java.io.*;
2 import java.util.ArrayList;
3
4 // Salvare ArrayList
5 ArrayList<Student> studenti = new ArrayList<>();
6 studenti.add(new Student("Ana", 20, 9.5));
7 studenti.add(new Student("Mihai", 22, 8.0));
8
9 try (ObjectOutputStream out = new ObjectOutputStream(
10     new FileOutputStream("studenti.ser"))) {
11     out.writeObject(studenti);
12 }
13
14 // Incarcare ArrayList
15 try (ObjectInputStream in = new ObjectInputStream(
16     new FileInputStream("studenti.ser"))) {
17     @SuppressWarnings("unchecked")
18     ArrayList<Student> lista = (ArrayList<Student>) in.readObject();
19 }
```

Condiție: Toate obiectele din colecție trebuie să fie Serializable!

Ce se serializează și ce nu?

Tip câmp	Serializat?	Valoare după deserializare
Câmpuri normale	✓ Da	Valoarea salvată
static	✗ Nu	Valoarea curentă din JVM
transient	✗ Nu	Valoare default (0, null, false)
Obiecte referite	✓ Da*	*Dacă sunt Serializable

Folosește **transient** pentru:

- Date sensibile (parole, chei)
- Conexiuni la baze de date
- Cache-uri care pot fi recalulate
- Orice nu are sens să fie salvat

Exemplu cu transient

```
1 public class Utilizator implements Serializable {
2     private static final long serialVersionUID = 1L;
3
4     private String username;
5     private transient String parola; // NU se salveaza!
6     private transient int loginCount; // NU se salveaza!
7
8     public Utilizator(String username, String parola) {
9         this.username = username;
10        this.parola = parola;
11        this.loginCount = 0;
12    }
13 }
```

După deserializare:

- username = valoarea salvată
- parola = null
- loginCount = 0

Obiecte compuse - toate trebuie Serializable

```
1  // CORECT - Adresa implementeaza Serializable
2  class Adresa implements Serializable {
3      private String oras;
4      private String strada;
5  }
6
7  class Persoana implements Serializable {
8      private String nume;
9      private Adresa adresa;  // OK - Adresa e Serializable
10 }
11
12 // GRESIT - va arunca NotSerializableException
13 class AdresaFaraSerializable {  // NU implementeaza Serializable
14     private String oras;
15 }
16
17 class Persoana2 implements Serializable {
18     private AdresaFaraSerializable adresa;  // EROARE la
19     serializare!
20 }
```

Exercițiu: transient vs static în serializare

Scop:

- să observăm ce se întâmplă cu:
 - variabilele `transient`
 - variabilele `static`
- după serializare + deserializare

Vom testa două variante:

- 1 NU modificăm valoarea variabilei statice
- 2 Modificăm valoarea variabilei statice înainte de deserializare

Clasa Utilizator (Serializable)

```
1 import java.io.Serializable;
2 public class Utilizator implements Serializable {
3     private static final long serialVersionUID = 1L;
4
5     private String username;
6     private transient String parola;    // NU se serializeaza
7     private transient int loginCount;  // NU se serializeaza
8     public static String aplicatie = "JavaApp"; // NU se
9                                           serializeaza
10    public Utilizator(String username, String parola) {
11        this.username = username;
12        this.parola = parola;
13        this.loginCount = 1;
14    }
15    @Override
16    public String toString() {
17        return "Utilizator{" + "username='" + username + '\'' +
18            ", parola='" + parola + '\'' +
19            ", loginCount=" + loginCount +
20            ", aplicatie='" + aplicatie + '\'' +
21            '}';
22    }
23 }
```

Pasul 1 — Serializarea obiectului

```
1 import java.io.*;
2 public class SerializareDemo {
3     public static void main(String[] args) throws Exception {
4
5         Utilizator u = new Utilizator("ana", "secret123");
6
7         System.out.println("INAINTE de serializare:");
8         System.out.println(u);
9
10        ObjectOutputStream out = new ObjectOutputStream(new
11            FileOutputStream("user.ser"));
12        out.writeObject(u);
13        out.close();
14    }
```

Varianța 1 — Static nemodificat

```
1  ObjectInputStream in=new ObjectInputStream(new FileInputStream("
    user.ser"));
2  Utilizator u2 = (Utilizator) in.readObject();
3  in.close();
4  System.out.println("DUPA deserializare:");
5  System.out.println(u2);
```

Output așteptat:

```
username='ana'
parola='null'
loginCount=0
aplicatie='JavaApp'
```

- **username** - a fost serializat
- **parola** - transient → devine null
- **loginCount** - transient → valoare implicită 0
- **aplicatie** - valoarea curentă a clasei

Important:

- Variabilele static NU aparțin obiectului
- Ele NU sunt serializate

Varianța 2 — Static modificat

```
1 public static void main(String[] args) throws Exception {
2     Utilizator u = new Utilizator("ana", "secret123");
3     System.out.println("INAINTE de serializare:");
4     System.out.println(u);
5     ObjectOutputStream out = new ObjectOutputStream(new
6         FileOutputStream("user.ser"));
7     out.writeObject(u);
8     out.close();
9     // Dupa serializare
10    Utilizator.aplicatie = "AltaAplicatie";
11    // Deserializare
12    ObjectInputStream in = new ObjectInputStream(new FileInputStream(
13        "user.ser"));
14    Utilizator u2 = (Utilizator) in.readObject();
15    in.close();
16    System.out.println(u2);
```

Output așteptat:

```
username='ana'
parola='null'
loginCount=0
aplicatie='AltaAplicatie'
```

Concluzii — transient vs static

- **transient**

- NU este serializat
- după deserializare → valoare implicită

- **static**

- aparține clasei, nu obiectului
- NU este serializat
- valoarea după deserializare este **valoarea curentă a clasei**

Regulă : *Serializarea salvează STAREA obiectului, nu a clasei*

Răspundeți **fără** a rula codul

Q1. Ce se întâmplă dacă parola NU mai este transient?

Q2. Ce se întâmplă dacă aplicație NU mai este static (ci devine un câmp normal al obiectului)?

Q3. Ce se întâmplă dacă adăugăm un constructor **fără parametri** care inițializează câmpurile?

Încercați să explicați DE CE, nu doar ce valoare apare.

Răspunsuri

R1. Dacă parola NU este transient:

- Valoarea parolei va fi **serializată**
- După deserializare, parola va avea **valoarea inițială**

R2. Dacă aplicație NU este static:

- Devine parte din obiect
- Este **serializată**
- După deserializare, va avea **valoarea din momentul serializării**

R3. Dacă există un constructor fără parametri:

- Constructorii NU sunt apelați la deserializare
- Valorile sunt citite direct din fișierul `.ser`

Deserializarea ocolește constructorii!

Exerciții 2

Exercițiul 1: Gestiune Studenți

Cerință: Creați o aplicație care:

- 1 Definește clasa `Student` (nume, grupa, notă) - serializabilă
- 2 Permite adăugarea de studenți de la tastatură
- 3 Salvează lista de studenți într-un fișier `.ser`
- 4 La repornire, încarcă studenții salvați anterior
- 5 Afișează studenții cu media peste o valoare citită

Hint-uri:

- Folosiți `Scanner` pentru citire de la tastatură
- `ArrayList<Student>` pentru stocare
- `ObjectOutputStream/ObjectInputStream` pentru persistență

Exercițiul 2: Procesare Fișier CSV

Cerință: Procesati un fișier `produse.csv`:

```
nume,pret,cantitate  
Laptop,3500.00,10  
Mouse,50.00,100  
Telefon,1050.00,80
```

- 1 Citiți fișierul linie cu linie
- 2 Creați obiecte `Produs` din fiecare linie
- 3 Calculați valoarea totală a stocului
- 4 Salvați produsele cu preț ≥ 100 într-un alt fișier

Hint-uri:

- `String.split(",")` pentru parsare
- `Double.parseDouble()` pentru conversie
- `BufferedReader` pentru citire, `PrintWriter` pentru scriere

Quiz 1: Streams

Întrebare: Ce tip de stream folosim pentru a citi un fișier imagine (.png)?

- ☐ A) FileReader
- ☐ B) BufferedReader
- ☐ C) FileInputStream
- ☐ D) Scanner

Răspuns: C) FileInputStream

Explicație: Imaginile sunt date binare, nu text. Reader/Writer sunt pentru caractere (text), iar InputStream/OutputStream sunt pentru bytes (binar).

Quiz 2: Ce afișează codul?

```
1 Scanner sc = new Scanner(System.in);
2 // Utilizatorul tasteaza: 25[Enter]Ana[Enter]
3 int varsta = sc.nextInt();
4 String nume = sc.nextLine();
5 System.out.println("Nume: [" + nume + "]");
```

- ☐ A) Nume: [Ana]
- ☐ B) Nume: []
- ☐ C) Nume: [Ana]
- ☐ D) Eroare la compilare

Răspuns: B) Nume: []

Explicație: `nextInt()` citește doar "25" și lasă "\n" în buffer.

`nextLine()` citește acel "\n" și returnează string gol. Soluție: adaugă `sc.nextLine();` după `nextInt()`.

Quiz 3: try-with-resources

Întrebare: Care afirmație despre try-with-resources este FALSĂ?

- ☐ A) Închide automat resursele la ieșirea din bloc
- ☐ B) Funcționează cu orice clasă care implementează AutoCloseable
- ☐ C) Poate avea mai multe resurse separate prin punct și virgulă
- ☐ D) Necesită obligatoriu un bloc catch

Răspuns: D) Necesită obligatoriu un bloc catch

Explicație: Blocul catch este opțional în try-with-resources. Poți avea doar try(...) { } fără catch, iar resursele tot se vor închide automat.

Quiz 4: Serializare

```
1 class Angajat implements Serializable {  
2     private String nume = "Ion";  
3     private transient String parola = "secret123";  
4     private static int counter = 100;  
5 }  
6 // Se serializeaza si deserializeaza un obiect Angajat
```

După deserializare, care sunt valorile câmpurilor?

- ☐ A) nume="Ion", parola="secret123", counter=100
- ☐ B) nume="Ion", parola=null, counter=100
- ☐ C) nume="Ion", parola=null, counter=0
- ☐ D) nume=null, parola=null, counter=0

Răspuns: B) nume="Ion", parola=null, counter=100

Explicație: transient → nu se salvează → devine null. static nu se serializează, dar păstrează valoarea din JVM.

Quiz 5: Ce excepție apare?

```
1 class Motor { } // NU implementeaza Serializable
2
3 class Masina implements Serializable {
4     private String marca = "Dacia";
5     private Motor motor = new Motor();
6 }
7
8 // Se incearca serializarea unui obiect Masina
9 ObjectOutputStream out = new ObjectOutputStream(
10     new FileOutputStream("masina.ser"));
11 out.writeObject(new Masina());
```

Răspuns: **NotSerializableException**

Explicație: Toate obiectele referite de un obiect serializabil trebuie să fie și ele Serializable. Motor nu este, deci serializarea eșuează.

Quiz 6: Alegerea clasei potrivite

Potrivește situația cu clasa recomandată:

1. Citire input de la tastatură	A. ObjectOutputStream
2. Citire fișier CSV mare	B. RandomAccessFile
3. Salvare obiect Java	C. Scanner
4. Editare byte la poziția 1000	D. BufferedReader

Răspunsuri:

- 1 → C (Scanner - are nextInt(), nextLine() etc.)
- 2 → D (BufferedReader - eficient pentru fișiere mari)
- 3 → A (ObjectOutputStream - serializare)
- 4 → B (RandomAccessFile - acces la poziție specifică)

Quiz 7: NIO.2

Întrebare: Care este echivalentul modern (Java 11+) al acestui cod?

```
1  BufferedReader br = new BufferedReader(new FileReader("date.txt"));
2  StringBuilder sb = new StringBuilder();
3  String line;
4  while ((line = br.readLine()) != null) {
5      sb.append(line).append("\n");
6  }
7  br.close();
8  String continut = sb.toString();
```

Răspuns:

```
String continut = Files.readString(Path.of("date.txt"));
```

O singură linie! NIO.2 simplifică dramatic codul pentru fișiere mici/medii.

Quiz 8: True or False?

Răspundeți cu Adevărat (A) sau Fals (F):

- 1 IOException este o checked exception
- 2 Scanner poate citi și din fișiere, nu doar de la tastatură
- 3 Constructorul este apelat la deserializare
- 4 FileWriter("f.txt", true) suprascrie fișierul
- 5 serialVersionUID este obligatoriu pentru Serializable

Răspunsuri:

- 1 **A** - trebuie tratată cu try-catch sau throws
- 2 **A** - Scanner(new File("..."))
- 3 **F** - obiectul e reconstruit fără constructor
- 4 **F** - true = append (adaugă la final)
- 5 **F** - recomandat, dar nu obligatoriu

Quiz 9: Debugging

Ce e greșit în acest cod?

```
1 public class Salvare {
2     public static void main(String[] args) {
3         try {
4             PrintWriter pw = new PrintWriter("log.txt");
5             pw.println("Mesaj important!");
6             // programul se termina aici
7         } catch (FileNotFoundException e) {
8             e.printStackTrace();
9         }
10    }
11 }
```

Problema: PrintWriter nu este închis!

Datele pot rămâne în buffer și să nu ajungă în fișier.

Soluție: Folosește try-with-resources:

```
try (PrintWriter pw = new PrintWriter("log.txt")) { ... }
```

Concluzii: Clase cheie Java I/O

Scop	Clase
Citire text	BufferedReader, Scanner, Files
Scriere text	PrintWriter, BufferedWriter, Files
Citire binară	FileInputStream, BufferedInputStream
Scriere binară	FileOutputStream, BufferedOutputStream
Serializare	ObjectOutputStream, ObjectInputStream
Lucru cu fișiere/directoare	File, Path, Files

Observații:

- 1 Folosiți **try-with-resources** pentru a închide automat fișierele.
- 2 Adaugați **Buffered*** pentru performanță
- 3 Tratați **excepțiile** specific (nu doar IOException generic)

Resurse suplimentare

- **Documentația oficială:**

- <https://docs.oracle.com/javase/tutorial/essential/io/>

- **Java NIO.2 (modern):**

- <https://docs.oracle.com/javase/tutorial/essential/io/fileio.html>

- **Serializare în profunzime:**

- <https://www.baeldung.com/java-serialization>